

SyncParty Project Specification

Zero-Cost, Multi-Room Real-Time YouTube Synchronized Watch Party Web Application

DOCUMENT VERSION

v1.0.0 (Production Blueprint)

INFRASTRUCTURE BUDGET

\$0.00 / MONTH (100% FREE TIER)

TARGET STACK

React / Tailwind CSS / Firebase

1. Executive Summary & Vision

SyncParty is a lightweight, serverless web application that enables users to create real-time virtual rooms where participants can watch and listen to YouTube content in synchronized real time. The primary objective is to deliver a seamless, low-latency group listening and watching experience with **zero operational infrastructure costs** for both the host and room visitors.

Prompt Engineering Purpose: This document is fully self-contained. It is structured to be fed directly into code generation LLMs (such as Cursor, Claude 3.5 Sonnet, or GPT-4o) to initiate full-stack scaffolding and implementation without requiring additional context.

2. Functional Feature Specifications

2.1 Room Management & Visitor Flow

- **Multi-Room Creation:** Any visitor can create an isolated watch party room with a single click or by navigating to a unique URL.
- **Dynamic Shareable Links:** Rooms are identified via URL parameters or hash fragments (e.g., `https://app.domain.com/?room=party-8f2a9c`). Visiting the link joins the room automatically.
- **Room Discovery / Lobby:** Visitors can view a public room selection list or enter a specific room ID to join existing parties.
- **One-Click Link Copy:** Built-in UI button to copy room invite links with animated toast/feedback indicators.

2.2 User Identity & Presence

- **Default Guest Usernames:** Auto-generate unique default usernames upon entry (e.g., `Guest_4821`) using client-side random generation and standard local storage persistence.
- **Profile Customization:** Users can change their display name at any point during a active session.
- **Real-time Presence List:** Live sidebar showing all active participants currently connected to the room.

2.3 Shared Queue & Playback Control

- **Shared Video Queue:** Any participant in the room can add valid YouTube video URLs or Video IDs to the synchronized playlist.
- **Role-Based Control Modes:**
 - *Open DJ Mode:* Any participant can pause, play, seek, or skip songs.
 - *Host DJ Mode:* Only the room creator controls timeline navigation, while guests auto-sync to host state.
- **Rejoin / Catch-Up Engine:** A explicit "Rejoin / Resync" button that calculates exact server time differences and forces local YouTube Player resynchronization.

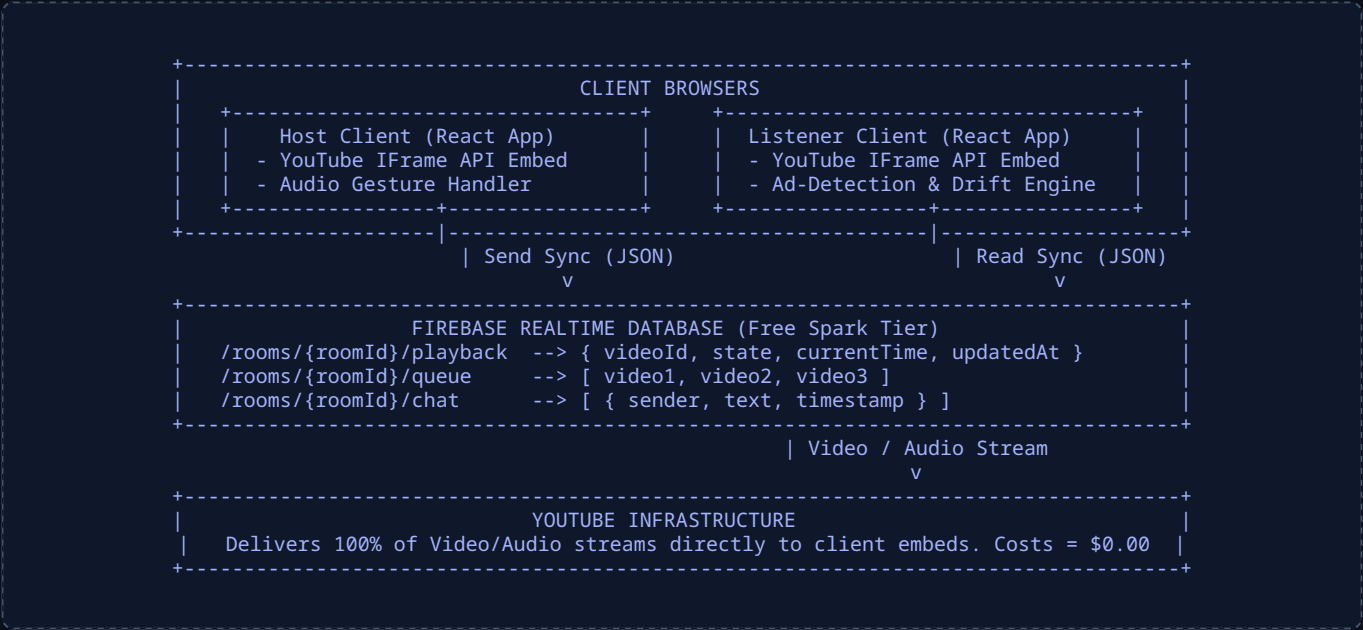
2.4 UI, Chat & Accessibility

- **Real-time Live Chat:** Lightweight messaging channel within each room tied to user identities.

- **Pop-Out / Mini Player UI:** A floating, absolute-positioned picture-in-picture style player box (^position: fixed; bottom: 20px; right: 20px) allowing users to interact with chat and queue while minimizing video size.
- **Background Playback Resilience:** Re-engagement notifications and audio gesture prompts for mobile web browsers that restrict background autoplay.

3. System Architecture & Cost Model

To ensure **ZERO HOSTING COSTS** at scale, SyncParty offloads media delivery entirely to YouTube's global CDN, using serverless Realtime DB WebSockets solely for low-bandwidth JSON metadata transmission.



3.1 Infrastructure & Platform Free Tier Analysis

Component	Selected Technology	Free Tier Provisioning	Cost
Static Web Hosting	Vercel / Cloudflare Pages / GitHub Pages	Unlimited bandwidth, Free SSL/TLS, Global Edge CDN	\$0.00
Realtime Database	Firebase Realtime Database (Spark Plan)	100 Concurrent Connections, 10 GB Data Transfer / month	\$0.00
Media Streaming	YouTube IFrame Player API Embeds	Unlimited video bandwidth handled directly by YouTube	\$0.00
Code Hosting & CI/CD	GitHub Public / Private Repository	Unlimited build minutes for public/private repos	\$0.00

4. Sync Logic, Ad & Network Drift Management

Critical Edge Case (YouTube Ads & Network Drift): Direct time syncing every fraction of a second causes video stuttering and breaks during unskipped YouTube ad breaks. The engine must adhere to **State-Based Elapsed Calculation**.

4.1 Expected Time Calculation Formula

Instead of streaming raw live timestamps, the Host publishes a snapshot containing `currentTime`, playback `state`, and `updatedAt` (Epoch timestamp). Listeners calculate expected timeline positioning dynamically:

$$\text{Expected Time} = \text{Host Timestamp} + (\text{Current Epoch Time} - \text{Host UpdatedAt Epoch})$$

4.2 Drift & Ad-Bypass Algorithm Rules

1. **Ad Exclusion Gate:** When a listener's local player is in State `-1` (Unstarted), `3` (Buffering for Ad), or playing an ad unit, incoming sync commands are ignored.
2. **Post-Ad Catchup:** Once the listener's local player transitions back to State `1` (Playing), calculate *Expected Time* and execute a single `player.seekTo()`.
3. **Drift Threshold:** Trigger `seekTo()` ONLY if $|\text{Local Time} - \text{Expected Time}| > 2.0 \text{ seconds}$. This prevents micro-stutter on minor latency fluctuations.

5. Code Architecture & Database Schema

5.1 Folder Structure (React / Modern JS)

```
src/
├── components/           # Reusable UI Components
│   ├── MiniPlayer.jsx   # Draggable / Corner Pop-Out Player
│   ├── QueueList.jsx    # Live Shared Playlist Queue
│   ├── ChatBox.jsx      # Live Chat Panel
│   └── RoomHeader.jsx    # Invite link copy & room state display
├── hooks/               # State & Synchronization Logic
│   ├── useYouTubePlayer.js # IFrame API wrapper & ad-drift algorithm
│   ├── useRoomSync.js     # Firebase Realtime DB listeners & state updater
│   └── useUserSession.js  # LocalStorage username & identity management
├── services/            # SDK Initializations
│   ├── firebase.js       # Database client configuration
│   └── youtubeApi.js     # Helper methods for YouTube metadata
├── utils/               # Math & Helper functions
│   └── syncMath.js       # Expected time formulas and drift calculations
└── App.jsx              # Main Router and Responsive Dashboard Shell
```

5.2 Firebase Realtime Database JSON Schema

```
{
  "rooms": {
    "party-8f2a9c": {
      "meta": {
        "createdAt": 1772000000000,
        "hostId": "usr_99a8b"
      },
      "playback": {
        "videoId": "dQw4w9WgXcQ",
        "state": 1,
        "currentTime": 42.5,
        "updatedAt": 1772000005120
      },
      "queue": [
        { "id": "q1", "videoId": "L_LUpnjgPso", "title": "Lofi Beat", "addedBy": "Guest_4821" }
      ],
      "users": {
        "usr_99a8b": { "name": "Host_Master", "joinedAt": 1772000000000 }
      },
      "messages": [
        { "id": "m1", "sender": "Guest_4821", "text": "Great song!", "timestamp": 1772000006000 }
      ]
    }
  }
}
```

6. AI-Guided Implementation Prompt Roadmap

Feed the following structured prompts sequentially into your target AI code generator:

Step 1: Scaffolding & Setup

"Initialize a React project using Vite and Tailwind CSS. Create a service module for Firebase Realtime Database using the provided JSON schema. Implement a custom hook `useUserSession` that generates or retrieves a default username like 'Guest_1234' from LocalStorage."

Step 2: Dynamic URL & Room Routing

"Create a `RoomHeader` component that parses the URL query parameter `?room=id`. If no ID is found, generate a random 6-character string (`party-xxxxxx`) and update the URL history. Include a 'Copy Invite Link' button with clipboard feedback."

Step 3: YouTube Sync Engine & Drift Control

"Write a React hook `useYouTubePlayer` that embeds the YouTube IFrame API. Implement synchronization logic using Firebase that calculates Expected Time = Host Time + (Now - Host UpdatedAt). Add a 2-second drift threshold and bypass sync commands while the local player is playing YouTube Ads."

